

Final Examination - ECE 124

Digital Circuits and Systems - Winter 2025

Instructor(s): Anwar Hasan

Date of exam: Thursday, April 17, 2025

Exam start time: 9:00 AM. Exam end time: 11:30 AM. Exam duration: 2.5 hours.

Exam type: Closed book. Materials allowed: None

Instructions:

- Write your Student ID number and Quest login userid in the space provided above if they are not already printed there.
- Answer all questions in this exam. Clearly state any assumptions you make when answering any questions. Show your work—no magic answers are acceptable.
- Both sides of each page are numbered. Your answer to a question must be in the space provided immediately below the question. For any overflow of answers, you may use the pages that have been marked for this purpose. Failure to follow this instruction properly can cause the answer to be not graded.

Properties of Boolean algebra (only a few of these are needed for answering questions in the exam):

- | | | |
|---|---|----------------|
| 1. (a) $x \cdot 0 = 0$ | (b) $x + 1 = 1$ | |
| 2. (a) $x \cdot 1 = x$ | (b) $x + 0 = x$ | |
| 3. (a) $x \cdot x = x$ | (b) $x + x = x$ | |
| 4. (a) $x \cdot x' = 0$ | (b) $x + x' = 1$ | |
| 5. (a) $(x')' = x$ | | |
| 6. (a) $x \cdot y = y \cdot x$ | (b) $x + y = y + x$ | (commutative) |
| 7. (a) $x \cdot (y \cdot z) = (x \cdot y) \cdot z$ | (b) $x + (y + z) = (x + y) + z$ | (associative) |
| 8. (a) $x \cdot (y + z) = x \cdot y + x \cdot z$ | (b) $x + y \cdot z = (x + y) \cdot (x + z)$ | (distributive) |
| 9. (a) $(x \cdot y)' = x' + y'$ | (b) $(x + y)' = x' \cdot y'$ | (DeMorgan) |
| 10. (a) $x + x \cdot y = x$ | (b) $x \cdot (x + y) = x$ | (absorption) |
| 11. (a) $x \cdot y + x \cdot y' = x$ | (b) $(x + y) \cdot (x + y') = x$ | (combining) |
| 12. (a) $x + x' \cdot y = x + y$ | (b) $x \cdot (x' + y) = x \cdot y$ | |
| 13. (a) $x \cdot y + y \cdot z + x' \cdot z = x \cdot y + x' \cdot z$ | (b) $(x + y) \cdot (y + z) \cdot (x' + z) = (x + y) \cdot (x' + z)$ | (consensus) |

Do **not** start a new answer on this page. Use this page only for any answer overflows from other pages.

1. [10 marks]

A three-bit up-down counter is controlled by two signals (labelled as c and d) and it produces a three-bit output ($z_2z_1z_0$). If $c = 0$, the counter does not count in any direction, neither up nor down. If $c = 1$ and $d = 0$, the counter adds 1 to its contents, i.e., acts as a normal up-counter. If $c = 1$ and $d = 1$, the counter subtracts 1 from its contents, acting as a normal down-counter. Assuming **synchronous** sequential circuits and the **Moore** model, draw a **state diagram** for the counter and give the corresponding **state table**. To receive the full mark, your answer must have no more than **eight** states. Draw your diagram neatly.

2. [10 marks]

The initial state table for an FSM is given below. Apply the **partitioning** technique to identify all the equivalent states, as defined below, and give the resulting **reduced state table**.

- [Two states are *equivalent* if for each input valuation their
- outputs are the *same* or *both unspecified*, and
 - next states are the *same*, *equivalent* or *both unspecified*.]

Present state	Next state		Output z	
	w=0	w=1	w=0	w=1
A	B	C	0	0
B	D	-	0	-
C	F	E	0	1
D	B	G	0	0
E	F	C	0	1
F	E	D	0	1
G	F	-	0	-

Do **not** start a new answer on this page. Use this page only for any answer overflows from other pages.

3. [10 marks]

Suppose your co-op employer has asked you to design an **asynchronous** sequential circuit to realize a two-bit up counter. Denoting the counter's (pulse) input as w , enable input as E , and output bits as z_1z_0 , give a **state diagram** of the machine and the corresponding **flow table**. Use the **Moore** model. To receive the full mark, your answer must have fewer than **ten** states. Draw your diagram neatly.

4. [10 marks]
- Below is a flow table for an FSM. Apply the **merging** procedure as many times as possible to derive the most reduced flow table for the FSM.

Present state	Next state					Output
	$w_2w_1 =$	00	01	10	11	
A		(A)	F	C	-	0
B		A	(B)	-	H	1
C		G	-	(C)	D	0
D		-	F	-	(D)	1
E		G	-	(E)	D	1
F		-	(F)	-	H	0
G		(G)	B	J	-	0
H		-	B	E	(H)	1
J		G	-	(J)	-	0

5. [10 marks]
- Below is a flow table for an FSM. Using **one-hot** encoding, find a race-free state assignment for the FSM and give the corresponding **excitation table**. To get the full mark, the output of any extra states that you use must have as many unspecified entries as possible.

Present state	Next state					Output z			
	$w_2w_1 =$	00	01	10	11	00	01	10	11
A		$\textcircled{A}$	B	D	$\textcircled{A}$	0	-	1	1
B		C	$\textcircled{B}$	$\textcircled{B}$	D	0	1	0	0
C		$\textcircled{C}$	$\textcircled{C}$	B	A	0	1	0	1
D		A	C	$\textcircled{D}$	$\textcircled{D}$	-	-	1	0

6. [10 marks]

a) $A=01110101$ and $B=11010110$ are two 8-bit 2's complement numbers. Perform $A-B$ using the two-step algorithm for 2's complement subtraction.

b) An n -bit ripple carry adder (RCA) can add two n -bit integers. The structure of an RCA is quite regular as it has n identical stages, each consisting of only a full adder (FA). The latter can add three bits (say, u, v , and w) to produce a sum bit s and a carry-out bit c satisfying the following “**arithmetic**” relationship: $u + v + w = 2c + s$, since c has twice the “weight” of s . See the diagram below for a FA block.

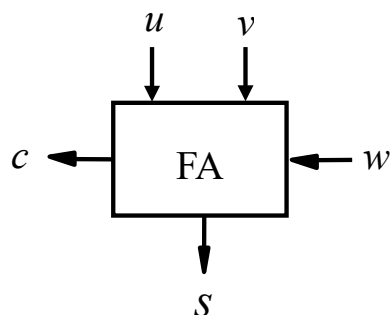
Suppose you want to extend the idea of RCA to build an arithmetic circuit that can add *three* n -bit integers, i.e., the new three-operand adder will consist of n identical stages. Denote the three operands as X, Y and Z . The i -th stage will add the following five inputs:

- i -th bits of the operands, i.e., x_i, y_i, z_i
- two carry-in bits: c_i from stage $i-1$ and d_i from stage $i-2$,

and will produce the following as output

- sum bit s_i
- two carry-out bits: c_{i+1} for stage $i+1$ and d_{i+2} for stage $i+2$

Noting that d_{i+2} has twice the weight of c_{i+1} and the latter has twice the weight of s_i , write down the **arithmetic** relationship between the inputs and outputs for the i -th stage. Design a circuit for the i -th stage of the three-operand adder using only FA blocks. To receive the full mark, your answer must have fewer than **four** FA blocks.



7. [10 marks]
- Consider the four-variable function $f(x_1, x_2, x_3, x_4) = \sum m(0, 3, 4, 5, 7, 9, 11) + D(8, 12, 13, 14)$, where x_1 represents the most significant bit, x_2 the second most significant bit, and so on. Populate the K-map given below and appropriately encompass 1's in the map to find all **PIs** and **EPIs** and give a **minimum cost SOP expression** for the function.

x_1x_2					
x_3x_4		00	01	11	10
	00				
	01				
	11				
	10				

8. [10 marks]

Consider functions f and g as given in the following two K-maps where empty cells imply to have 0's. It has already been determined that the SOP implementations of f and g , when considered individually, have a minimum cost of 15 and 19, respectively. Your task is to determine another set of SOP expressions for these functions with one or more common product terms that can be shared between the implementations of f and g so that the overall cost is lower than 34 (=15+19). Use the following K-maps to **derive your low-cost expressions** and then **draw the circuit diagrams**. Assume that the input variables in both true and complemented forms are available free of cost.

		x_1x_2			
		00	01	11	10
x_3x_4	00	1	1		1
	01			d	1
	11	d	d	1	d
	10		1		

Function f

		x_1x_2			
		00	01	11	10
x_3x_4	00	1	1	d	d
	01		1	d	1
	11	1	1		1
	10			d	

Function g

Do **not** start a new answer on this page. Use this page only for any answer overflows from other pages.

9. [10 marks]

Consider the four-variable function $f(x_1, x_2, x_3, x_4) = \sum m(0, 1, 5, 6, 12, 13, 14) + D(2, 3, 7, 11)$, where x_1 represents the most significant bit, x_2 the second most significant bit, and so on. Using the **tabular method**, i) generate all the prime implicants (PIs) and ii) find any essential PIs (EPIs) that may exist.

Do **not** start a new answer on this page. Use this page only for any answer overflows from other pages.

10. [10 marks]

For a 4-variable function $f(x_1, x_2, x_3, x_4)$, below is its prime implicant (PI) cover table after the EPIs are removed. Use the concept of **row/column dominance**, if applicable, and **Petrick’s method** to determine the necessary non-EPIs needed for a minimum cost cover for the function.

PIs	Minterms				
	m_3	m_5	m_7	m_9	m_{11}
$p_1=0x11$	✓		✓		
$p_2=x011$	✓				✓
$p_3=01x1$		✓	✓		
$p_4=10x1$				✓	✓
$p_5=x10x$		✓			
$p_6=1x0x$				✓	

Do **not** start a new answer on this page. Use this page only for any answer overflows from other pages.